a single run. If the same test is repeated independently and a number is reported as prime each time, the error probability decreases multiplicatively. For example, running the test twice reduces the error probability to 1/4 × 1/4 = 1/16. By repeating the test many times, the probability of mistakenly accepting a composite number as prime can be made arbitrarily small, though never exactly zero. Importantly, these algorithms are designed so that they never classify a true prime as composite; the only possible error is falsely labelling a composite number as prime.

Let us now examine how SageMath allows the user to choose between deterministic and probabilistic primality testing algorithms. When the function *is_prime( )* is used in SageMath, either a deterministic or a probabilistic algorithm is selected internally, depending on the size of the number being tested and the level of confidence required in the result. The SageMath program 'primality47.sage' (line numbers included for clarity), shown below, demonstrates how the programmer can control the algorithm used in the background for primality testing.

```
1. n = random_prime(2^1024)
2. %time n.is_prime(proof=False)
3. print(n.is_prime( proof=False))
4. %time n.is_prime(proof=True)
5. print(n.is_prime(proof=True))
6. n = 100
7. %time n.is_prime(proof=False)
8. print(n.is_prime(proof=False))
9. %time n.is_prime(proof=True)
10. print(n.is_prime(proof=True))
```

Now, let us try to understand the working of the program. Line 1 generates a random prime number less than 2^{1024}. Notice that this is a very large (approximately 1024-bit) prime number. When Line 2 is executed, SageMath delegates the computation to the PARI/GP library. PARI/GP runs the Baillie–PSW primality test (a combination of the Miller–Rabin and Strong Lucas primality testing algorithms) to determine whether the given number is prime. If the algorithm returns *False*, the number is composite (with 100% certainty). If it returns *True*, the number is classified as probably prime—that is, it is not a proven prime, and there remains a very small probability that it could be composite. The magic command *%time* measures how long the computation takes. Line 3 prints the result of the probabilistic test. Line 4 runs a deterministic primality testing algorithm because the option *proof=True* is specified. In this case, SageMath uses either the APR-CL test (Adleman–Pomerance–Rumely–Cohen–Lenstra) or ECPP (Elliptic Curve Primality Proving). We will not discuss these algorithms here, as they are advanced topics beyond the scope of this series. The time taken for this operation is also measured using %time. Line 5 prints the result of the deterministic test. Lines 6 to 10 perform the same operations on a small composite number (in this case, 100).

When the program 'primality47.sage' is executed, the output shown in Figure 4 is obtained. From the first six lines of the output, we can verify that the large number is correctly identified as prime by both the probabilistic and deterministic algorithms. The timing data also shows that the probabilistic algorithm is approximately 791 times faster than the deterministic algorithm for this large input. Lines 7 to 12 confirm that both algorithms correctly classify the number 100 as composite. Interestingly, in this case the deterministic algorithm outperforms the probabilistic one. This highlights an important practical observation: for relatively small inputs, deterministic algorithms may be preferable. They can be faster in this situation and, moreover, provide a guaranteed correct result without relying on probabilistic confidence.

```
CPU times: user 4.46 ms, sys: 0 ns, total: 4.46 ms
Wall time: 4.55 ms
True
CPU times: user 3.63 s, sys: 25.7 ms, total: 3.65 s
Wall time: 3.6 s
True
CPU times: user 13 µs, sys: 0 ns, total: 13 µs
Wall time: 16.7 µs
False
CPU times: user 9 µs, sys: 0 ns, total: 9 µs
Wall time: 11.4 µs
False
```

Figure 4: Comparison of deterministic and probabilistic primality tests

Now it is time to wrap up our discussion. In this article, we have explored several important topics in computer science that are rooted in number theory. Precise integer and floating-point operations are essential in nearly all real-world applications. Even small computational errors can lead to catastrophic consequences. For example, the failure of the Ariane 5 rocket launched by Arianespace on June 4,1996, was caused by an integer overflow error, resulting in a loss of more than US$ 370 million. This highlights the critical importance of accurate numerical computation. Likewise, primality testing plays a vital role in many real-world applications, particularly in areas such as cryptography and security. Given the fundamental importance of number theory, we will explore it further in the next article in this series. END

By: Dr Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science. He is an assistant professor (senior scale) at Manipal Institute of Technology, Bengaluru, and maintains a technical blog.